LuminaInc / **jina-embeddings-v3** like 0 Follow Lumina AI Inc 2

Feature Extraction | Transformers | PyTorch | ONNX | Safetensors | sentence-transformers | 94 languages | xlm-roberta | sentence-similarity | mteb | custom_code | Eval Results | text-embeddings-inference | arxiv:2409.10173 | License: cc-by-nc-4.0

Train | Deploy | Use this model

Model card | Files xet | Community

Downloads last month
-

Safetensors Model size 572M params | Tensor type BF16 | Files info

Inference Providers NEW

Feature Extraction

This model isn't deployed by any Inference Provider. Ask for provider support

Evaluation results

cosine_pearson on MTEB AFQMC (default)	validation set	self-reported	41.742
cosine_spearman on MTEB AFQMC (default)	validation set	self-reported	43.473
euclidean_pearson on MTEB AFQMC (default)	validation set	self-reported	42.245
euclidean_spearman on MTEB AFQMC (default)	validation set	self-reported	43.525
main_score on MTEB AFQMC (default)	validation set	self-reported	43.473
manhattan_pearson on MTEB AFQMC (default)	validation set	self-reported	42.046
manhattan_spearman on MTEB AFQMC (default)	validation set	self-reported	43.309
pearson on MTEB AFQMC (default)	validation set	self-reported	41.742
spearman on MTEB AFQMC (default)	validation set	self-reported	43.473
main_score on MTEB ArguAna-PL (default)	test set	self-reported	50.118

Expand 11129 evaluations

The embedding model trained by Jina AI.

jina-embeddings-v3: Multilingual Embeddings With Task LoRA

Quick Start

Blog | Azure | AWS SageMaker | API

Intended Usage & Model Info

`jina-embeddings-v3` is a **multilingual multi-task text embedding model** designed for a variety of NLP applications. Based on the Jina-XLM-RoBERTa architecture, this model supports Rotary Position Embeddings to handle long input sequences up to **8192 tokens**. Additionally, it features 5 LoRA adapters to generate task-specific embeddings efficiently.

Key Features:

- **Extended Sequence Length:** Supports up to 8192 tokens with RoPE.
- **Task-Specific Embedding:** Customize embeddings through the `task` argument with the following options:
  - `retrieval.query`: Used for query embeddings in asymmetric retrieval tasks
  - `retrieval.passage`: Used for passage embeddings in asymmetric retrieval tasks

- `separation`: Used for embeddings in clustering and re-ranking applications
- `classification`: Used for embeddings in classification tasks
- `text-matching`: Used for embeddings in tasks that quantify similarity between two texts, such as STS or symmetric retrieval tasks

- **Matryoshka Embeddings**: Supports flexible embedding sizes (`32, 64, 128, 256, 512, 768, 1024`), allowing for truncating embeddings to fit your application.

Supported Languages:

While the foundation model supports 100 languages, we've focused our tuning efforts on the following 30 languages: **Arabic, Bengali, Chinese, Danish, Dutch, English, Finnish, French, Georgian, German, Greek, Hindi, Indonesian, Italian, Japanese, Korean, Latvian, Norwegian, Polish, Portuguese, Romanian, Russian, Slovak, Spanish, Swedish, Thai, Turkish, Ukrainian, Urdu,** and **Vietnamese.**

> *"⚠️ **Important Notice:***
> *We fixed a bug in the `encode` function #60 where **Matryoshka embedding truncation** occurred after normalization, leading to non-normalized truncated embeddings. This issue has been resolved in the latest code revision.*
>
> *If you have encoded data using the previous version and wish to maintain consistency, please use the specific code revision when loading the model: `AutoModel.from_pretrained('jinaai/jina-embeddings-v3', code_revision='da863dd04a4e5dce6814c6625adfba87b83838aa', ...)`"*

Usage

▶ **Apply mean pooling when integrating the model.**

The easiest way to start using `jina-embeddings-v3` is with the Jina Embedding API.

Alternatively, you can use `jina-embeddings-v3` directly via Transformers package:

```
!pip install transformers torch einops
!pip install 'numpy<2'
```

If you run it on a GPU that support FlashAttention-2. By 2024.9.12, it supports Ampere, Ada, or Hopper GPUs (e.g., A100, RTX 3090, RTX 4090, H100),

```
!pip install flash-attn --no-build-isolation
```

```
from transformers import AutoModel

# Initialize the model
model = AutoModel.from_pretrained("jinaai/jina-embeddings-v3", trust_remote_code=T

texts = [
    "Follow the white rabbit.",  # English
    "Sigue al conejo blanco.",  # Spanish
    "Suis le lapin blanc.",  # French
    "跟着白兔走。",  # Chinese
    "اتبع الأرنب الأبيض.",  # Arabic
    "Folge dem weißen Kaninchen.",  # German
]

# When calling the `encode` function, you can choose a `task` based on the use cas
# 'retrieval.query', 'retrieval.passage', 'separation', 'classification', 'text-ma
# Alternatively, you can choose not to pass a `task`, and no specific LoRA adapte
embeddings = model.encode(texts, task="text-matching")

# Compute similarities
print(embeddings[0] @ embeddings[1].T)
```

By default, the model supports a maximum sequence length of 8192 tokens. However, if you want to truncate your input texts to a shorter length, you can pass the `max_length` parameter to the `encode` function:

```
embeddings = model.encode(["Very long ... document"], max_length=2048)
```

In case you want to use **Matryoshka embeddings** and switch to a different dimension, you can adjust it by passing the `truncate_dim` parameter to the `encode` function:

```
embeddings = model.encode(['Sample text'], truncate_dim=256)
```

The latest version (3.1.0) of SentenceTransformers also supports `jina-embeddings-v3`:

```
!pip install -U sentence-transformers
```

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("jinaai/jina-embeddings-v3", trust_remote_code=True)

task = "retrieval.query"
embeddings = model.encode(
    ["What is the weather like in Berlin today?"],
    task=task,
    prompt_name=task,
)
```

You can fine-tune `jina-embeddings-v3` using SentenceTransformerTrainer. To fine-tune for a specific task, you should set the task before passing the model to the ST Trainer, either during initialization:

```
model = SentenceTransformer("jinaai/jina-embeddings-v3", trust_remote_code=True, m
```

Or afterwards:

```
model = SentenceTransformer("jinaai/jina-embeddings-v3", trust_remote_code=True)
model[0].default_task = 'classification'
```

This way you can fine-tune the LoRA adapter for the chosen task.

However, If you want to fine-tune the entire model, make sure the main parameters are set as trainable when loading the model:

```
model = SentenceTransformer("jinaai/jina-embeddings-v3", trust_remote_code=True, m
```

This will allow fine-tuning the whole model instead of just the LoRA adapters.

▶ **ONNX Inference.**

Contact

Join our Discord community and chat with other community members about ideas.

License

`jina-embeddings-v3` is listed on AWS & Azure. If you need to use it beyond those platforms or on-premises within your company, note that the models is licensed under CC BY-NC 4.0. For commercial usage inquiries, feel free to contact us.

Citation

If you find `jina-embeddings-v3` useful in your research, please cite the following paper:

```
@misc{sturua2024jinaembeddingsv3multilingualembeddingstask,
      title={jina-embeddings-v3: Multilingual Embeddings With Task LoRA},
      author={Saba Sturua and Isabelle Mohr and Mohammad Kalim Akram and Michael G
      year={2024},
      eprint={2409.10173},
      archivePrefix={arXiv},
      primaryClass={cs.CL},
      url={https://arxiv.org/abs/2409.10173},
}
```